

ถ้าระบบรองรับ Promotion แบบซ้อนกัน ต้องจัดลำดับคำนวณยังไง

1. ถ้าระบบรองรับ Promotion แบบซ้อนกัน ต้องจัดลำดับคำนวณยังไง

ควรกำหนด **Calculation Order** ให้ชัดเจน และใช้ **priority** ควบคุมลำดับภายในแต่ละกลุ่มลำดับที่แนะนำคือ:

ITEM → CART → COUPON → SHIPPING → ROUNDING

ความหมาย:

ลำดับ	Scope	ตัวอย่าง
1	ITEM	Product 1 ลด 10%, Product 2 ลด 100 บาท
2	CART	ซื้อครบ 1,000 ลด 100
3	COUPON	ใช้โค้ดลดเพิ่ม
4	SHIPPING	ส่งฟรี
5	ROUNDING	ปัดเศษราคา

สำหรับโจทย์นี้มี promotion ต่อสินค้าเป็นหลัก ดังนั้นเริ่มจาก **ITEM** ก่อน เช่น

Product 1 ราคา 1,000 ลด 10%
→ เหลือ 900

Product 2 ราคา 500 ลด 100
→ เหลือ 400

จากนั้นค่อยเอายอดหลัง item discount ไปรวมเป็น cart subtotal

Cart subtotal หลัง item discount = 900 + 400 = 1,300

ถ้ามี cart-level promotion ค่อยคำนวณต่อจากยอดนี้

Rule สำหรับ Promotion ซ้อนกัน

ควรมี field ควบคุมดังนี้:

Field	ใช้ทำอะไร
scope	แยกว่าเป็น item/cart/coupon/shipping
priority	ลำดับการคำนวณ เลขน้อยทำก่อน
stackable	ใช้ร่วมกับ promotion อื่นได้หรือไม่
exclusive	ถ้าใช้ตัวนี้แล้ว ห้ามใช้ตัวอื่นที่ conflict
stop_processing	ถ้าใช้ตัวนี้แล้ว ให้หยุดคำนวณ promotion ถัดไป
max_discount_amount	จำกัดส่วนลดสูงสุด
conflict_group	กลุ่ม promotion ที่ห้ามใช้รวมกัน

หลักการป้องกันลดราคาซ้ำผิดพลาด

ใช้สูตรนี้ทุกครั้ง:

```
actual_discount = min(calculated_discount, remaining_amount)
```

และต้องรับประกันว่า:

```
final_price >= 0
```

ตัวอย่าง:

```
สินค้า 100 บาท  
Promotion A ลด 80 บาท  
Promotion B ลด 50 บาท
```

```
ถ้าใช้ซ้อนกัน:  
หลัง A เหลือ 20  
B ลดได้จริงแค่ 20  
Final = 0  
ไม่ใช่ -30
```

2. ถ้าต้องการเพิ่ม Promotion จะทำอย่างไรเพื่อไม่ให้กระทบ Logic เดิม

ถ้าเป็น promotion ที่ระบบรองรับอยู่แล้ว เช่น

- ลดเป็นเปอร์เซ็นต์
- ลดเป็นจำนวนเงิน
- ใช้กับสินค้า
- ใช้กับ category
- ใช้เมื่อยอดถึงขั้นต่ำ

ไม่ควรแก้ code หลัก แต่ให้เพิ่มข้อมูลใน database แทน

ตัวอย่างเพิ่ม promotion:

```
Product 3 ลด 15%
```

เพิ่มข้อมูลประมาณนี้:

```
promotions
```

```
code = PRODUCT_3_15_PERCENT
scope = ITEM
priority = 10
stackable = true
status = ACTIVE
```

```
promotion_targets
```

```
target_type = PRODUCT
target_id = 3
```

```
promotion_actions
```

```
action_type = PERCENTAGE_DISCOUNT
value_percent = 15
applies_to = ITEM
```

ระบบเดิมจะทำงานได้ทันที เพราะ Promotion Engine อ่าน config จาก DB แล้วเลือก Strategy ตาม `action_type`

หลักการออกแบบ

เพิ่ม promotion ปกติ = เพิ่ม config ใน DB
ไม่ต้องแก้ calculation engine

นี่คือการออกแบบตาม **Open/Closed Principle**

Open for extension
Closed for modification

คือระบบเปิดให้เพิ่มพฤติกรรมใหม่ได้ แต่ไม่ควรต้องแก้ logic เดิมทุกครั้ง

3. ถ้าต้องการเพิ่ม Promotion ใหม่ที่ไม่เคยมีมาก่อน จะ Design ยังไงให้รองรับ

ถ้า promotion ใหม่เป็นแบบที่ระบบยังไม่เคยรองรับ เช่น

ซื้อ 2 แกม 1
ซื้อสินค้า A แล้วลดสินค้า B
ลดตาม Tier
ลดเฉพาะลูกค้ากลุ่ม VIP

ให้เพิ่มเป็น **Strategy ใหม่** แทนการแก้ logic กลาง

แนวทางคือ:

1. เพิ่ม `action_type` ใหม่
2. เพิ่ม configuration ใน `promotion_actions.value_json`
3. สร้าง Promotion Strategy ใหม่
4. Register Strategy เข้า Strategy Registry
5. เพิ่ม test case
6. ไม่แก้ core calculation flow

ตัวอย่าง:

```
action_type = BUY_X_GET_Y
value_json = {
  "buyQuantity": 2,
  "getQuantity": 1,
  "discountPercent": 100
}
```

จากนั้นเพิ่ม strategy:

```
BuyXGetYStrategy
```

Promotion Engine ยังทำงานแบบเดิม:

```
Load Promotion
→ Check Condition
→ Sort
→ Find Strategy by action_type
→ Apply Strategy
→ Generate Breakdown
```

สิ่งที่เปลี่ยนคือเพิ่ม Strategy ใหม่เข้าไปเท่านั้น

ทำไมวิธีนี้ดี

เพราะหลีกเลี่ยง code แบบนี้:

```
if promotion_type == percentage
else if promotion_type == fixed
else if promotion_type == buy_x_get_y
else if promotion_type == tier_discount
...
```

ถ้าเขียนแบบนี้ระบบจะโตยาก แก่ยาก และเสี่ยงทำ logic เดิมพัง

คำตอบสรุป

ระบบควรจัดลำดับการคำนวณ promotion ด้วย `scope` และ `priority` โดยแนะนำลำดับ
`ITEM → CART → COUPON → SHIPPING → ROUNDING` และใช้ `stackable`, `exclusive`, `stop_processing`,

`conflict_group` เพื่อควบคุม promotion ซ้อนกัน

ถ้าต้องเพิ่ม promotion ที่ระบบรองรับอยู่แล้ว ให้เพิ่มเป็น configuration ใน database ผ่าน `promotions` , `promotion_targets` , `promotion_conditions` , `promotion_actions` โดยไม่ต้องแก้ core logic

ถ้าต้องเพิ่ม promotion รูปแบบใหม่ที่ไม่เคยมีมาก่อน ให้เพิ่ม `action_type` ใหม่ และสร้าง `Strategy` ใหม่ แล้ว register เข้า Promotion Engine เพื่อให้ระบบขยายได้ตามหลัก **Open/Closed Principle** โดยไม่กระทบ logic เดิม